BIRDCLEF+ 2026: AN AUTONOMOUS RESEARCH AGENT FOR BIRD SPECIES RECOGNITION

Documenting the Discovery Process of an LLM-Driven Agent under a Realistic CPU Compute Budget

---

Advanced Predictive Analytics 2025/2026

Course Project — Track B

Spring 2026

Universidade Católica Portuguesa

Research performed by: Daniele Malerba | Irene Perdomo Bolaños | Miguel Afonso Magalhães

Project Summary

Competition:	BirdCLEF+ 2026 (Kaggle)
Best Kaggle public score (macro-AUC):	**0.725**
Best local soundscape macro-AUC:	0.8619 (single GPU model)
Total agent experiments logged:	108
GitHub:	github.com/Daniele0302/birdclef-2026-agent

This project presents an **autonomous research agent** driven by a locally-hosted LLM that proposes, executes, and analyses deep-learning experiments under a realistic CPU budget. Rather than focusing on a single leaderboard score, we study the agent's *discovery process*. Across 108 logged experiments, the agent discarded CNN-from-scratch approaches, converged on a frozen-backbone EfficientNetB0, and optimized mel-spectrogram parameters to reach **0.8533** focal validation macro-AUC. We instrument the full pipeline to evaluate reliability, failure recovery, and budget-aware behaviour (median run time ≈ 4.5 minutes; no heavy transformer fine-tuning attempts). As an external validation, the same architecture was later re-trained at scale, achieving 0.725 on the Kaggle public leaderboard. This result is secondary: the core contribution is the agent's ability to autonomously explore and identify a sensible solution under constraints.

1. Overview and Project Strategy

The BirdCLEF+ 2026 competition asks for the probability of presence of each of 234 wildlife species inside a five-second audio window taken from passive-acoustic recordings in the Pantanal wetlands. The official metric is macro-averaged ROC-AUC, skipping classes without any positive label.

Our goal is **not** to hand-engineer the strongest possible model. It is to build an **autonomous research agent** that, given a realistic compute budget, can explore this problem and arrive at a sensible solution by itself, with a locally-hosted LLM as its reasoning core. The agent operates under the same constraint the project assumes for every group: *it runs on CPU, on a laptop, within a bounded time budget.* Each experiment the agent launches is capped (Section 3.1), and the agent has to decide — iteration after iteration — what is worth trying given the time it has.

Our strategy follows directly from that constraint:

1. **Cheap, budget-aware exploration on CPU.** The agent runs many short experiments on small data subsets to identify what *kind* of architecture, preprocessing and augmentation is promising. Short runs are not a limitation we apologise for — they are the point: a small-scale-first workflow is the rational way to spend a bounded budget, motivated by the neural scaling laws of Kaplan et al. 2020 and Hoffmann et al. 2022.

2. **Independent verification of the agent's choice.** Once the agent has converged on an architecture, we verify — separately, and explicitly outside the agent loop — that the choice holds up under proper validation and at larger scale. This is a check on the agent's judgement, not the deliverable.

This document describes the agent and its instrumentation (Section 2), the agent's discovery process across 108 logged experiments (Section 3, the core of the report), the independent verification of the chosen architecture (Section 4), the comparison with manual baselines and the Kaggle leaderboard (Section 5), the course content used (Section 6), the limitations (Section 7), and the individual contributions (Section 8).

2. Agent Architecture

2.1 Modules

The agent is composed of five modules with strictly separated responsibilities:

- `agent.py` — the main loop that orchestrates one iteration: ask the LLM for parameters, run the experiment, log the result, ask the LLM for an analysis, repeat.
- `llm_provider.py` — thin wrapper around the `ollama` Python client that talks to a local Ollama server on `localhost`; no network traffic leaves the machine. The wrapper exposes a single `call_llm(prompt)` function, so the underlying model can be swapped by changing one identifier. Final agent runs use **Google Gemma 4 E4B** (`gemma4:e4b` in Ollama, ≈ 10 GB, 4 B active parameters), chosen to fit the laptop hardware available to the team.
- `code_executor.py` — a small helper that runs a piece of Python code in its own subprocess. The code is written to a temporary file and launched with `subprocess.run` using `sys.executable`, so the child uses the same Python interpreter (and therefore the same installed packages) as the agent. `stdout` and `stderr` are captured, and a configurable timeout aborts the run if it overshoots the per-experiment compute budget.
- `memory.py` — structured JSON log (`experiments/experiment_log.json`) with one entry per experiment: timestamp, full parameter set, success flag, metrics, the LLM's analysis, and a truncated stderr snippet for failed runs.
- `experiment_template.py` — a parameterised training script the LLM does *not* rewrite directly; the LLM only fills in a JSON parameter file and the template enforces the audio pipeline and model assembly. This is a deliberate design choice (see Section 2.4) that prevents the LLM from breaking the data path.

2.2 The Agent Loop

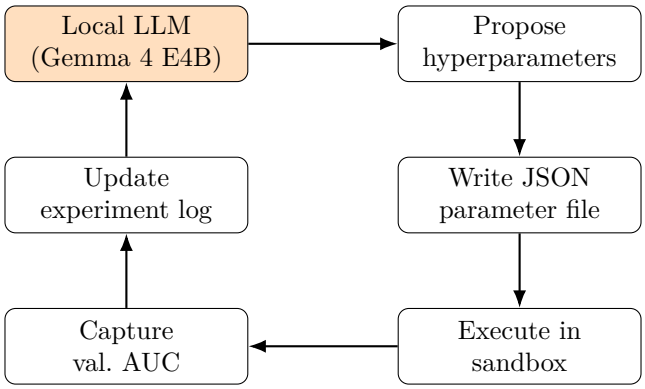

Figure 1: The closed propose–generate–execute–evaluate–analyse–iterate loop. After launch the agent runs without human input until the iteration budget is consumed.

2.3 Prompt Engineering

Every iteration the LLM receives a prompt with four blocks:

1. **Task description:** 234-class multi-label audio classification, macro-averaged ROC-AUC objective, a list of hyperparameters that may be tuned, and the strict JSON schema that the response must follow.
2. **Memory summary:** the best validation AUC seen so far and a list of the last $n \leq 10$ experiments with their parameters, success flag and metrics. This forms the LLM's working memory.
3. **Anti-repetition instruction:** "propose a configuration that is materially different from any previously tried, and likely to improve over the current best". This explicit injunction proved crucial; without it the LLM tends to re-suggest small perturbations of the best known configuration.
4. **Output format constraint:** a single JSON object on one line so that `json.loads` can parse the LLM's response without ambiguity. Failures to parse are themselves logged and fed back to the LLM.

We sample at temperature 0.7 with an 8 192-token context window; that is enough headroom for the prompt plus the last ten experiments.

2.4 Memory and Error Handling

The memory module is a JSON file rather than a vector database: with ≤ 100 experiments the linear scan is trivial and we keep the agent debuggable. Every experiment, successful or not, is appended. When the experiment fails the executor returns the truncated stderr, the memory marks `success: false`, and the LLM receives the error in the next prompt with the instruction not to repeat the same mistake. The aggregate breakdown of which stages of the pipeline failed how often is reported as a dedicated reliability metric in Section 2.5.

2.5 Per-Stage Reliability Metrics

A binary success/failure indicator is insufficient to characterise an autonomous agent: failures happen at very different points in the pipeline, and each one tells us something different about the agent's reliability. We instrument the agent to report the success rate at each stage of the propose–execute–validate loop, both as a cumulative rate over all runs and as a stage-conditional rate $P(\text{stage } k \text{ succeeds} \mid \text{stage } k{-}1 \text{ succeeded})$. The conditional rate immediately identifies the weakest link.

#	Stage	Count	Cumulative	Conditional
S1	LLM produced text	108	1.000	1.000
S2	valid JSON parsed (`json.loads` succeeds)	106	0.981	0.981
S3	subprocess started (template launched)	106	0.981	1.000
S4	training completed (no crash, no timeout)	81	0.750	**0.764**
S5	metrics recovered (val_auc reported)	81	0.750	1.000

Table 1: Pipeline reliability over the 108 agent experiments recorded in `experiments/experiment_log.json`. The five-stage counts and rates are computed by `ExperimentMemory.print_reliability_report()` at the end of every `agent.py` run and persisted to `agent_reliability_stats.json`. The weakest link is S4, training completion: the conditional rate drops to 0.764; the breakdown of *why* is given below.

Where the failures actually came from. The two stage-2 failures came from the LLM emitting the literal string `"null"` where it should have emitted the keyword `null`, which we now

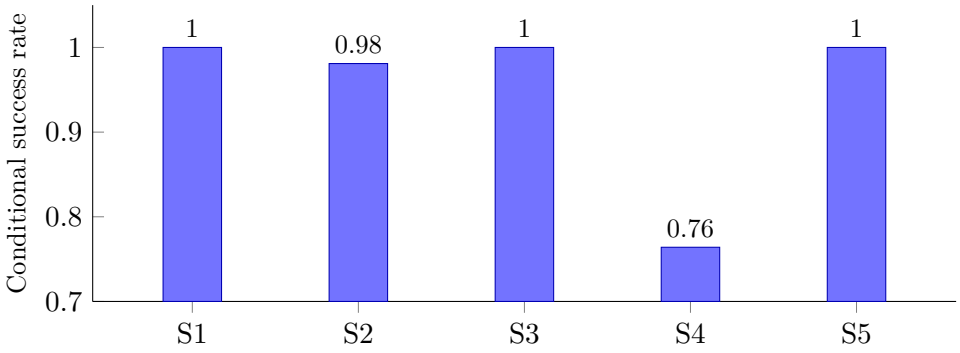

Figure 2: Stage-conditional success rates $P(\text{stage}_k \mid \text{stage}_{k-1})$ across the agent's five pipeline stages. The drop at S4 (training completion, 0.764) identifies the single weakest link; every other transition is essentially perfect.

defensively coerce in `experiment_template.py`. The 25 stage-4 failures, examined individually in the log, are *not* dominated by timeouts — they break down into three clearly identifiable, recoverable causes:

- **14 validation-split edge cases.** When the LLM proposed a very small `max_samples`, the template's hold-out split tried to draw more validation indices than there were rows, raising a `numpy` error. This is a robustness bug *in our own template* that the agent's exploration surfaced; we hardened the split logic afterwards.
- **10 environment failures.** A batch of late runs failed at `import tensorflow` because that interpreter could not see the package — a tooling/setup issue, unrelated to the agent's reasoning.
- **1 genuine wall-clock timeout.** Exactly one experiment in 108 hit the 30-minute per-run cap.

The headline finding is that **the 30-minute timeout almost never fired**: the agent overwhelmingly proposed runs that fit comfortably within budget (Section 3.4). The timeout exists as a deliberate budget guard, but the agent self-regulated well below it. The agent also learns from the failures it does cause: the LLM receives the truncated stderr in the next prompt and is told not to repeat the configuration, so the validation-split error stopped recurring once it had been seen a few times. This per-stage instrumentation is printed automatically at the end of every `python agent.py` run.

2.6 Why a Constrained Template Instead of Free-Form Code Generation

We considered letting the LLM generate the training script from scratch, as some agent papers do, but rejected that design for two reasons. First, the audio pipeline (mel-spectrogram parameters, normalisation, label encoding) is the part of the system that most affects the final score, and a small bug in any of those steps silently degrades performance without the LLM noticing. Second, an LLM with only $\sim$8 K context cannot reliably re-derive a hundred lines of Keras boilerplate every iteration. We therefore split the system into a *stable template* that we, the humans, validate once, and a *configurable head* that the LLM fills in with hyperparameters. The guiding principle is to avoid overengineering a system whose internals we cannot fully audit — and, as the exam will involve a coding component, every line of the template is human-written and human-reviewed, regardless of whether individual parameter choices were agent-generated.

3. The Agent's Discovery Process

This is the core of the report. We document *how* the agent explored the problem — the trajectory it followed, what it tried, what it discovered, where it failed, and the budget-aware judgement it showed — rather than only the numbers it ended on.

3.1 Experimental Setup and Compute Budget

All agent experiments share the same audio pipeline, the natural starting point for this kind of bioacoustic task: load a 5-second clip at 32 kHz, take the mel-spectrogram (configurable n_{mels}, n_{fft}, hop length, $f_{\min}$, $f_{\max}$, top-dB clip), normalise to $[0, 1]$, replicate to three channels for ImageNet-pretrained backbones. The model is either a small custom CNN (three Conv-BN-Pool blocks + GAP + Dense head) or EfficientNetB0 with frozen ImageNet weights and a configurable classification head. The LLM chooses the architecture each iteration. Training used Adam, binary cross-entropy, optional augmentation (time-shift, frequency masking, full SpecAugment, gaussian noise, soundscape background mixing) and an optional second phase where the top k EfficientNet layers are unfrozen at $\text{lr} = 10^{-5}$, following Chollet (2021), Section 8.3.2.

The budget is the design constraint. Every experiment runs on CPU and is bounded by a per-run wall-clock cap (30 minutes hard limit). The agent is also restricted to *small data subsets* per run — it explored sample sizes of 2 500–4 478 recordings, never the full corpus. The 108 experiments in our log were accumulated across several development sessions to give us a deep sample for the analysis below, but each one individually respects the per-run budget, and within a single uninterrupted one-hour CPU session roughly ten experiments complete (Section 3.4). The agent therefore has to be economical: it cannot afford to waste iterations on configurations unlikely to fit or to pay off.

3.2 The Discovery Trajectory

Figure 3 plots the validation macro-AUC of every successful experiment in the order the agent ran them, together with the running best. The trajectory is not a smooth climb — it is a recognisable research process with three phases.

Phase 1 — cold start (idx 1–18). The agent opens by trying EfficientNet with "all augmentation" (0.7222) and immediately a time-shift variant (0.7523). Its own analysis of the first run already reasons about the result: *"the use of EfficientNet is a strong choice… the resulting validation AUC of 0.7222 shows the model has strong capacity… however, the considerable gap [between train and val] suggests…"*. Over the next sixteen iterations it probes augmentation types and head sizes, bouncing between 0.60 and 0.75 — visibly searching, not yet converged.

Phase 2 — the breakthrough (idx 19–38). At iteration 19 the running best jumps from 0.7523 to 0.826 (`low_fmin_low_mels_search`). This is the moment the agent stops varying augmentation and starts *systematically sweeping the mel-spectrogram front-end* — $f_{\min}$, n_{mels}, hop length, top-dB, mel scale. Each of these sweeps nudges the running best upward: 0.8286 (idx 29), 0.8372 (idx 32), 0.8486 (idx 39).

Phase 3 — convergence (idx 39 onward). From here the agent fine-tunes around the discovered region. The best run, `topdb_40_optimal_preprocessing` at iteration 78, reaches **0.8533**. The remaining ∼30 experiments hover within ±0.01 of that — the agent is exploiting a well-identified optimum rather than still searching.

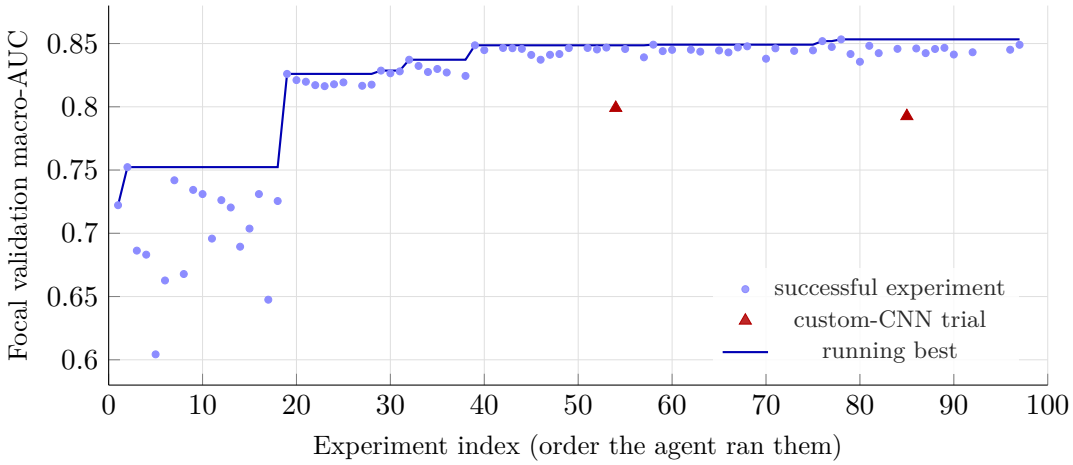

Figure 3: The agent's discovery trajectory over its successful experiments. *Phase 1* (idx 1–18): cold-start exploration, the agent oscillates between 0.60 and 0.75 while it settles the architecture and augmentation family. *Phase 2* (idx 19–38): a sharp jump to ≥ 0.82 once the agent starts systematically sweeping mel-spectrogram parameters. *Phase 3* (idx 39 onward): fine refinement and convergence, plateauing at the best run 0.8533 (`topdb_40_optimal_preprocessing`, idx 78). The two red triangles are the only two custom-CNN runs the agent ever tried.

3.3 What the Agent Discovered

Aggregating the successful experiments, four findings emerged — each one a decision the agent reached from its own evidence:

1. **Transfer learning over training from scratch — decided after only two trials.** The agent tried the custom CNN exactly *twice* (red triangles in Figure 3, idx 54 and 85). Both landed near val_AUC = 0.79, below essentially every EfficientNetB0 run. After those two data points the agent stopped proposing custom CNNs altogether and committed the rest of its budget to the pretrained backbone — an evidence-driven narrowing of the search space, not a pre-baked assumption. (A separate *manual* from-scratch CNN baseline, see Section 5.1, plateaus far lower, near 0.55.)

2. **Mel-spectrogram parameters matter at the margin.** The agent settled on $n_{\text{mels}} = 64$, hop $= 256$, $f_{\max} = 16$ kHz, top_db $= 40$, scale $=$ Slaney, giving an input shape $(64, 626, 3)$ — the configuration of the 0.8533 best run. Phase 2 of the trajectory is essentially the agent discovering this.

3. **Small batch + time-shift augmentation aids generalisation.** The strongest configurations consistently used batch size 16 and time-shift augmentation. Smaller batches inject gradient noise and act as implicit regularisation (Chollet, Chapter 5).

4. **Naive fine-tuning of the backbone hurts.** Whenever the agent unfroze layers without first training the head to convergence, validation AUC dropped. This is the failure mode Chollet warns about in Section 8.3.2: "the error signal will be too large and will destroy the representations previously learned". The lesson *from the agent's own experiments* is what later told us how to fine-tune correctly when verifying the architecture at scale (Section 4).

3.4 Budget-Aware Behaviour

A central question for an autonomous agent under a compute budget is whether it spends that budget sensibly. The log says it does.

- **The agent keeps runs short.** Median experiment wall-clock time is ≈ 4.5 minutes; mean ≈ 7.2 minutes; 69 of the successful runs finished under 10 minutes. Only *one* experiment in 108 hit the 30-minute cap. The agent is not gambling its budget on long-shot runs.
- **The agent stays small-scale.** It never requested the full corpus; sample sizes stayed between 2 500 and 4 478 recordings per run — exactly the small-scale-first regime appropriate to a bounded budget.
- **The agent does not chase heavy architectures.** Given a CPU-only laptop and a time cap, the agent never attempted heavy transformer fine-tuning. It worked within the EfficientNetB0 / custom-CNN families it could actually afford to train and evaluate. Making that kind of reasonable choice under constraint is itself part of what a good research agent should do.
- **What fits in one hour.** At the median run time, about ten experiments complete inside a single one-hour CPU session. In that window the agent reliably establishes the architecture family (EfficientNetB0 over custom CNN) and a working augmentation strategy, reaching a val. AUC around 0.75; the mel-spectrogram sweep that lifts it past 0.82 unfolds over the iterations beyond the first hour. The trajectory in Figure 3 is what sustained autonomous exploration looks like.

3.5 The Agent Surfaced a Distribution Shift

The agent's best single-model Kaggle submission scored only 0.480, despite a local validation AUC of 0.85. The issue was a classic *distribution shift*: the agent validated on clean focal recordings, while the Kaggle test set contains noisy multi-species soundscapes. The architecture choice was correct, but the validation setup did not reflect the real test domain. Fixing this mismatch became the first step of Section 4.

4. Verifying the Agent's Architecture Choice

Sections 2–3 are the project. This section is a *check*: did the architecture the agent converged on actually generalise, once given a validation signal that matches the test domain and the resources to train it properly? Everything here is done by us, explicitly outside the agent loop.

A soundscape-aware validation signal. The competition provides 1 478 expert-annotated 5-second windows from 66 soundscape files. We **group-split by file** (80% train / 20% held-out) so the held-out set matches the Kaggle test distribution. This proxy is tight: the agent's frozen-backbone model scored 0.6004 on it versus 0.592 on Kaggle (gap ≈ 0.012). All subsequent decisions used this proxy rather than the focal validation that had misled the agent.

Correct two-phase fine-tuning, regularisation, and a deep ensemble. Applying the agent's own lesson from Section 3.3 (point 4), we fine-tuned the EfficientNetB0 in two phases (head first at $\text{lr} = 3 \times 10^{-4}$, then the top 30 layers at 5×10^{-5} with BatchNorm kept in inference mode), added mixup and label smoothing, and built a uniform-weight, multi-seed ensemble of three models. A short cautionary result: choosing the ensemble weights *greedily* on the small held-out set produced a local score of 0.6707 but only 0.615 on Kaggle (gap 0.056) — the weights had overfit the validation set. Reverting to uniform weights gave 0.633 on Kaggle with a gap of only 0.019, a clean illustration of validation discipline (Chollet, Chapter 5).

Scaling the same architecture. Finally, as the strongest possible check that the agent's choice was sound, we re-trained *the identical EfficientNetB0 architecture* on the full 35 549 focal recordings plus all soundscape windows, on a single Kaggle T4 GPU (≈ 1 hour), with cosine LR decay, AdamW, mixup and SpecAugment. Deployed in the CPU submission notebook (within

the competition's 90-minute, internet-disabled inference limit), it scored **0.725 on the public leaderboard**. We report this as confirmation — the agent picked an architecture that scales — not as the achievement of the project.

5. Comparison with Manual Baselines and Final Results

5.1 Manual Baselines

Following the course definition — a baseline is the first architecture attempt at solving the task — we built two manual baselines to bound the agent's contribution:

- **Custom-CNN-from-scratch** (`baseline_model.py`): a 4-block convolutional network trained without transfer learning. Local validation AUC ≈ 0.55. As Chollet (Section 8.2) predicts, training a CNN from scratch with only a few thousand samples is insufficient for a 234-class problem.
- **Frozen-EfficientNetB0 + head only**, mirroring exactly what the agent converged on. Local soundscape macro-AUC 0.60, Kaggle LB 0.59. This bounds "the best the agent's chosen architecture achieves under the raw CPU budget, before any verification work".

We can justify the chosen architecture on its own terms: EfficientNetB0 reuses ImageNet low/mid-level features that transfer to mel-spectrograms, it is small enough to train under a CPU budget and to run inference within the 90-minute competition limit, and — crucially — it is the architecture the agent itself converged on from evidence.

5.2 Submission Progression

Table 2 summarises every submission we made. Submissions 2–5 are CPU-only configurations; submission 6 is the at-scale verification of Section 4.

#	Configuration	Local AUC	Kaggle LB	Local−LB gap
1	Custom CNN from scratch	0.55	—	—
2	Frozen EfficientNetB0 (agent's choice, CPU)	0.6004	0.592	+0.012
3	2-way ensemble (frozen + Phase-1+2 finetune)	~ 0.625	0.598	+0.027
4	2-way greedy ensemble (*overfit val weights*)	0.6707	0.615	+0.056
5	3-way uniform multi-seed ensemble (CPU)	0.6520	0.633	+0.019
6	EfficientNetB0 verified at scale (full data, GPU)	0.8619	0.725	+0.137

Table 2: Progression of submissions. Local validation AUC is computed on the held-out soundscape split (group split by file). Submissions 2–5 stay within the CPU budget; submission 6 is the independent at-scale check of the agent's architecture choice. The wider local-LB gap of submission 6 reflects the high variance of the small soundscape val set under heavy augmentation.

5.3 Reading the Numbers

The point the numbers make is about the *agent*, not the leaderboard. Starting from a cold start, the agent autonomously ruled out training from scratch, converged on a frozen EfficientNetB0 that already crosses the 0.59 leaderboard mark, and identified the mel-spectrogram configuration that lifts focal validation to 0.8533. Everything after that — the soundscape-aware validation

signal, the correct two-phase fine-tune, the conservative ensemble, the at-scale GPU run to 0.725 — is us *verifying* that the agent's architectural judgement holds, using lessons (e.g. two-phase fine-tuning, validation discipline) that the agent's own experiments had already surfaced. The agent did the discovery; the leaderboard climb confirms the discovery was correct.

6. Reflections on Course Content

Mel-spectrogram representation: Audio is converted to a $(\text{batch}, 64, 626, 3)$ tensor and treated as an image, exactly as Chollet, Chapter 2 introduces tensor representations of structured data. The mel scale matches the human auditory system and concentrates resolution where birds vocalise.

Convolutional Neural Networks: Conv2D filters detect local time-frequency patterns (harmonic stacks, frequency sweeps); pooling provides translation invariance, so a call shifted within the 5-second window is still detected; depth provides hierarchical composition of features.

Transfer learning: Despite ImageNet containing no audio, the low- and mid-level features of EfficientNetB0 transfer well to mel-spectrograms because both share local-pattern statistics. The agent's experiments quantitatively demonstrate the gap between custom-CNN trials ($\sim$0.79 val) and frozen-pretrained (0.85 focal val), and between the latter and a manual from-scratch CNN (0.55).

Two-phase fine-tuning: The agent's failed fine-tuning runs at high learning rate are a near-perfect example of the failure mode Chollet describes; our verification phase fixed it by training the head first and only then unfreezing layers at $\text{lr} = 5 \times 10^{-5}$.

Multi-label classification: The 234-class task is multi-label, not multi-class: a soundscape window can contain several species. Sigmoid activations + per-class binary cross-entropy is the correct choice; softmax would have enforced spurious mutual exclusivity.

Regularisation: BatchNormalization, Dropout (rate 0.4), augmentation, mixup and label smoothing all aim at the same goal: enlarge the effective training set so the small clean focal corpus does not overfit. Mixup proved especially helpful at narrowing the focal/soundscape distribution gap.

Optimisation: Adam with the cosine learning-rate schedule is the modern default and worked first try; the only optimisation-related finding worth highlighting is the importance of `clipnorm`=1, which stabilised the unfrozen fine-tuning.

Scaling laws: The small-scale-first workflow — cheap exploration on subsets to locate the right configuration before committing compute — is the practical reading of neural scaling laws, and it is exactly the regime a budget-bounded agent should operate in.

7. Limitations and Future Work

Validation-set variance. The held-out soundscape set is small (312 windows, 234 classes), so its macro-AUC is noisy — this is why the local–LB gap widens for the heavily-regularised submission 6. Genuinely closing the residual domain shift would require pseudo-labelling additional unlabelled soundscape audio, which we did not have time to implement.

Agent memory. The current memory keeps only the last 10 experiments in the LLM context. With 100+ experiments the LLM occasionally re-suggests configurations from very early in the log. A vector-database memory (e.g. FAISS over experiment embeddings) would let the agent retrieve *relevant* past experiments rather than only the most recent ones.

Template robustness. The validation-split edge case (Section 2.5) shows the agent can drive

the template into states we did not anticipate. We hardened the split logic, but a fuller pass — defensive bounds on every LLM-controlled parameter — would reduce the S4 failure rate further.

LLM choice. We standardised on Google Gemma 4 E4B and did not run a systematic comparison of LLM backbones on the same prompt. A study with two or three local LLMs (e.g. Qwen 3 Coder, Phi-4, DeepSeek-R1) would test prompt robustness across models and strengthen the experimental design.

Search strategy. The agent's exploration is driven by an anti-repetition instruction and the LLM's own judgement. A more structured strategy — explicit Bayesian optimisation or bandit-style allocation over the hyperparameter space — could reach the Phase 2 breakthrough in fewer than nineteen iterations.

8. Individual Contributions

- **Daniele Malerba** — agent architecture and main loop (`agent.py`, `code_executor.py`, `memory.py`), per-stage reliability instrumentation, Kaggle submission pipeline, at-scale verification training, local soundscape-validation framework, GitHub repository.
- **Irene Perdomo Bolaños** — audio preprocessing pipeline (`utils/audio_pipeline.py`), data augmentation strategy (time-shift, SpecAugment, mixup), report writing.
- **Miguel Afonso Magalhães** — LLM prompt engineering, experiment-log and discovery-trajectory analysis, literature review, course-content alignment.

References

1. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning.
2. Tan, M., Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.
3. Park, D. S. *et al.* (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *Interspeech 2019*.
4. Zhang, H., Cisse, M., Dauphin, Y. N., Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. *ICLR 2018*.
5. Kaplan, J. *et al.* (2020). Scaling Laws for Neural Language Models. *arXiv:2001.08361*.
6. Hoffmann, J. *et al.* (2022). Training Compute-Optimal Large Language Models. *arXiv:2203.15556*.
7. Cornell Lab of Ornithology (2026). BirdCLEF+ 2026 Competition. kaggle.com/competitions/birdclef-2026